Chapter 2 解答

CH02-Q01

Register 編號為 `$t0=8`、`$t1=9`、`$s0=16`、`$sp=29`。

1. `sll` 是 R-type，opcode 為 0。欄位為

```
op=000000  rs=00000  rt=01001  rd=01000  shamt=00011  funct=000000
```

組合後

$$I = (9 \ll 16) + (8 \ll 11) + (3 \ll 6) = 0x00090000 + 0x00004000 + 0x000000c0 = 0x000940c0.$$

2. `addiu` 是 I-type，opcode 為 9，immediate 是 `-16 = 0xfff0`。

$$I = (9 \ll 26) + (29 \ll 21) + (16 \ll 16) + 0xfff0 = 0x27b0fff0.$$

3. 第一題是 R-format，opcode 0，以 `funct` 選擇 shift operation，`shamt` 是 unsigned shift amount。第二題是 I-format，opcode 9，immediate 在 arithmetic 解釋時先 sign-extend 成 32-bit `0xfffffff0`。兩者 instruction alignment 都使 full word 是 4 bytes。

CH02-Q02

1. `0xfffffff6` 的 unsigned 值為

$$15(16^7) + 15(16^6) + \cdots + 6 = 4294967286.$$

最高 bit 為 1，signed two's complement 值為

$$4294967286 - 2^{32} = -10.$$

2. 數學和的低 32 bits 是 `0x80000000`，但正數相加已超過 signed 範圍。MIPS `add` 觸發 overflow exception，目的暫存器不寫入結果；`addu` 不 trap，才會寫入該低位元。

3. `0xffffffff + 1` 產生 bit pattern `0x00000000`，carry-out 為 1，所以 unsigned overflow 成立。Signed 解釋下 $-1 + 1 = 0$，沒有 signed overflow。

4. 單看 bit pattern 不能決定比較結果。若程式契約把 `0xffffffff` 解讀成 unsigned counter，應使用 `sltu`，值 4294967295 大於 1；若解讀成 signed balance，應使用 `slt`，值 -1 小於 1。先有型別與資料語意，才有對應的 compare instruction。

CH02-Q03

1. 每次進入 loop body 時，令 $i = R[t0]$。不變量為

$$0 \le i \le n, \qquad R[v0] \equiv \sum_{k=0}^{i-1} |a[k]| \pmod{2^{32}},$$

其中只對可完成 `sub` 的元素使用 modulo 加總；`0x80000000` 會在第 3 小題觸發 exceptional case。

2. 初始化令 $R[t0] = 0$、$R[v0] = 0$。若 $i < n$，`beq` 不跳躍，delay slot 的 `nop` 先執行，body 處理 `a[i]`，最後 `addiu` 建立 $i + 1$。若 $i = n$，`beq` 在執行 `nop` 後前往 `done`。因此 body 依序在 $i = 0, 1, \ldots, n-1$ 執行，恰好 $n = R[a1]$ 次；`n=0` 時為 0 次。

例如 `a=[-5,3]`，初始暫存器 `(t0,v0)=(0,0)`：

進入 body 時	讀入 `$t2`	處理後 `$v0`	下一輪 `$t0`
0	-5	5	1
1	3	8	2

第三次 branch 條件 $2 = 2$ 成立而離開；實際 body 執行 2 次。

3. `0x80000000` 是最小 signed integer。`sub $t2,$zero,$t2` 所求的 mathematical absolute value 是 2^{31}，超出 signed 32-bit 可表示上限，因此觸發 signed overflow exception，不寫入運算結果。若改用 `subu`，則不 trap，modulo result 為 `0x80000000`，仍不代表正的 2^{31}。程式必須先定義飽和、64-bit accumulator 或錯誤處理。

4. `addu` 不因 signed overflow trap，而是保留 modulo 2^{32} result；若程式需要精確總和，必須使用更寬 accumulator、檢查 carry 或限定輸入。`sub` 在取負數絕對值時仍可能 trap，不能由後面的 `addu` 修正已發生的例外。

CH02-Q04

1. `outer`會修改`$s0`，須保存呼叫者的舊值；nested `jal`會覆寫`$ra`，所以也保存進入outer時的返回位址。若某個caller-saved暫存器或argument在inner返回後仍存活，也須由outer保存。本例用callee-saved `$s0`保存傳入的唯一argument，不借用題目未定義的`$s1/$s2`。
2. 配置32-byte frame，0..15保留為outgoing argument home area，20..23可作額外local slot；24..27保存舊s0、28..31保存舊ra。假設傳入sp依此ABI已8-byte aligned，減32後仍對齊：

```
addiu $sp, $sp, -32
sw    $ra, 28($sp)
sw    $s0, 24($sp)
addu  $s0, $a0, $zero
```

先把incoming argument放進s0；若outer先有計算，可在這裡以已定義的輸入更新s0。本例把保存的同一argument傳給inner：

```
addu  $a0, $s0, $zero
jal   inner
nop
```

`jal`後ra是該call的PC+8；inner依約保留s0，返回outer的continuation。若inner回傳值位於v0，outer可在恢復自身狀態前使用它；不可假設t/a registers仍保留call前值。Exit：

```
lw    $s0, 24($sp)
lw    $ra, 28($sp)
addiu $sp, $sp, 32
jr    $ra
nop
```

先完成固定offset的loads再釋放frame；同時存活的資料不能占同一slot。兩個restore的先後不必反向，只要地址與資料相依正確且sp尚未釋放。

3. 兩次sequential calls可重用同一個32-byte frame及outgoing area，不需每次再減32。進入outer的ra已在28(sp)保存一次即可；每個jal自己的continuation由當次ra交給inner。若第一次的v0或其他caller-saved值還要跨第二次call使用，須另存到20(sp)等不重疊local slot，或另用已保存的callee-saved register；需要更多同時存活值時才擴frame。
4. inner破壞s0而未恢復，違反callee-saved契約。outer作為inner的caller可依賴s0保留；outer自己返回時也須把24(sp)的舊s0恢復給更外層caller。

CH02-Q05

1. Immediate `0xfffa` sign-extend 為 -6，並左移 2：

$$\Delta = -6 \times 4 = -24 = 0xffffffe8.$$

因此

$$target = 0x00400010 - 0x18 = 0x003fffff8.$$

2. J-type 取 next PC 的 upper four bits `0x0`，再接 26-bit target 與 `00`：

$$target = (0x0 \ll 28) \vee (0x0004000 \ll 2) = 0x0010000.$$

3. 目標 `0x1abcdef4` 的 upper four bits 是 `0x1`，但 next PC 的 upper four bits 是 `0x0`，所以不能由這個 J instruction 直接到達。指令只能保留同一個 256-MiB region 的 upper bits；跨 region 需要 indirect jump、trampoline 或其他 sequence。
4. Offset 2 代表 `2×4=8` bytes。

CH02-Q06

1. 普通load/add/store不是一個atomic RMW：兩core可同讀x，各自寫x+1，最後只增一次。LL/SC retry使只有成功的conditional store算一次完成；兩core都最終成功各一次時，counter才增加兩次（按32-bit modulo表示）。
2. `ll`讀值並建立reservation；`sc`以rt舊值作store data。失敗時不寫shared address、rt變0；成功時寫入新counter、rt變1。失敗後retry會重新ll，而不是把狀態值0再加1直接store。另一core此時可能已更動memory，所以失敗不代表memory仍是本core最初讀到的值。
3. 題設已明定 reservation 失效，因此 `sc` 不寫入共享位址並回傳 0；其他 core 仍可能已改變記憶體。具體哪些事件使 reservation 失效與監控範圍須依 ISA 版本和平台確認，不能只憑「插入 store」一詞概括。
4. Atomicity不保證starvation freedom；某core可反覆失敗。若要求進展，需要另外給出fair scheduling、競爭限制或backoff等機制及其進展條件，不能由單條atomic instruction推出有界等待。

CH02-Q07

1. Assembler 把 pseudo-instructions、local labels 與 instruction encodings 固定，對 external `dot` 與 `axpy` 留下 relocation or symbol reference。Static linker 合併 `main.o` 與 `math.o`，解析 `dot`，並為 `axpy` 建立 dynamic dependency 與 PLT/GOT 或等價 relocation。Dynamic loader 在 process 啟動時映射 shared library、解析 runtime symbols，處理 base address 與 lazy/eager binding。
2. Object sections 以 0 為起點，最終載入位址尚未確定。PC-relative branch、absolute address、GOT entry 與 data pointer 需要 linker 按 relocation type、symbol value 與 addend 修正。沒有 relocation record，linker 無法區分普通的 32-bit immediate 和需要修正的 address。
3. File offset 是 executable 在檔案中的 byte 位置。Virtual address 是 process 載入後 CPU 使用的 address space 座標。Loader 依 segment headers 把 file ranges 映射到 virtual ranges；兩者可以有不同起點、alignment 與 page 組合。
4. PC-relative call 的 offset 欄位可能有 signed range。若 library 映射太遠，直接 call 的 displacement 可能無法 encode，需要 linker producer thunk/PLT，再由 indirect jump 進入 library。即使 system 使用一般 PC-relative sequence，toolchain 也可能重排 code layout。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CH02-Q08

1. Version A：

$$T_A = \frac{(1.8 \times 10^9)(1.20)}{3.0 \times 10^9} = 0.720 \text{ s}.$$

Version B：

$$T_B = \frac{(1.5 \times 10^9)(1.55)}{3.0 \times 10^9} = 0.775 \text{ s}.$$

2. $S_{B/A} = T_A/T_B = 0.720/0.775 \approx 0.929$。B 較慢，速度約為 A 的 0.929，或執行時間增加約 7.64%。
3. Reduction 是 $(1.8 - 1.5)/1.8 = 1/6 \approx 16.7\%$，但 CPU time 還乘上 average CPI。B 的每 instruction work 與 time 使 CPI 從 1.20 增至 1.55，增加 29.2%，足以超過 instruction-count 的 gain。
4. 要求 $T_B = T_A = 0.720$ s。因

$$T_B = \frac{(1.5 \times 10^9)(1.55)}{f_B},$$

解得

$$f_B = \frac{(1.5 \times 10^9)(1.55)}{0.720} \approx 3.22917 \text{ GHz}.$$

這個答案只在一階固定 CPI 假設下成立。Frequency 提升可能改變 CPI、power 與 memory stalls。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH02-Q09

1. `0x8fa8000c` 的 `op=0x23=35`，是 I-format `lw`；`rs=29=$sp`、`rt=8=$t0`、`imm16=12`。Effective address 是 `0x0000000c + R[sp]`，以 modulo 2^{32} 計算，且 word address 必須 4-byte aligned。
2. `0x1109fffc` 的 `op=4`、`rs=8=$t0`、`rt=9=$t1`、`imm16=0xfffc=-4`。它是 `beq $t0,$t1,-4`。`PCnext=0x00400024`，target 是

$$0x00400024 + (-4 \times 4) = 0x00400014.$$

Taken sequence 是 `0x00400020`、`0x00400024`、`0x00400014`；not taken 是 `0x00400020`、`0x00400024`、`0x00400028`。中間的 `0x00400024` 是 delay slot。

3. `op=3`、`target26=0x100140`，所以 target 的低 28 bits 是 `0x00400500`，再沿用 `PCnext=0x00400074` 的高四位 0，得到 `0x00400500`。`$ra=PC+8=0x00400078`，delay slot 在 `0x00400074`。
4. 若誤把 `-4` 當 byte displacement，target 會錯誤算成 `0x00400020`。真正的 branch displacement 必須先左移 2，單位是 instruction。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CH02-Q10

1. `id` 位於 offset 0 到 7；`vals` 位於 offset 8 到 23；`sum` 位於 offset 24 到 31。最大 alignment 是 8，因此 `sizeof(struct row)=32`。`rows[5].vals[3]` 的地址為

$$0x3000 + 5 \times 32 + 8 + 3 \times 4 = 0x3000 + 160 + 8 + 12 = 0x30b4.$$

2. 令 `%rdx` 保存base。有效sequence為 `movslq %esi,%rsi`、`salq $5,%rdi`、`leaq 8(%rdi,%rsi,4),%rax`，最後 `leaq (%rdx,%rax),%rax`。前一條lea只算offset 0xb4；後一條加base 0x3000，才得到完整address 0x30b4；`lea` 讀取的是 register addressing expression，不送出 memory read request。
3. 單一元素視為已排序。`i=1,j=1` 比較 `3>1`，交換後 `[1,3,2]`，再令 `j=0` 結束。`i=2,j=2` 比較 `3>2`，交換後 `[1,2,3]`；`j=1` 比較 `1>2` 不成立而停止。
4. 7 與 5 只含等價的 loop body、loop test 與一個 branch delay slot；不含 function prologue、初始 guard、最終 exit control、cache miss、branch misprediction 或 exception。Compiler 的 strength reduction 可把 `i*4` 改成 pointer increment，induction-variable elimination 可把固定 base addition 提到 loop 外，兩份原始碼就可能有相同 execution count。

參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 2. CSO course slides, Chapter 2.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study